

Exercise 5 - Sampling Intervals for Models

Florescu Andrei - 12242987

2024

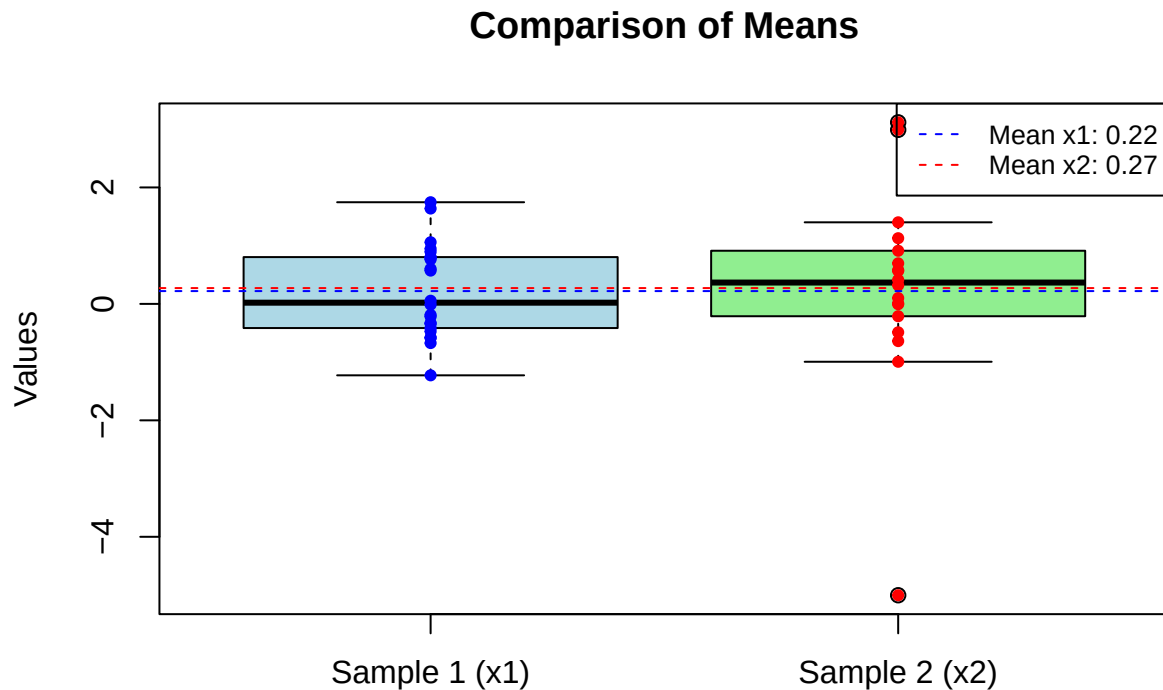
Task 1

```
set.seed(12242987)
x1 <- c(-0.673, -0.584, 0.572, -0.341, -0.218, 0.603, -0.415, -0.013, 0.763, 0.804, 0.054, 1.746, -0.473)
x2 <- c(0.913, -0.639, 2.99, -5.004, 3.118, 0.1, 1.128, 0.579, 0.32, -0.488, -0.994, -0.212, 0.413, 1.404)
```

1. Plot the data in a way which visualises the comparison of means appropriately.

```
mean_x1 <- mean(x1)
mean_x2 <- mean(x2)

boxplot(list(x1 = x1, x2 = x2),
  col = c("lightblue", "lightgreen"),
  names = c("Sample 1 (x1)", "Sample 2 (x2)"), main = "Comparison of Means",
  ylab = "Values"
)
points(rep(1, length(x1)), x1, col = "blue", pch = 19, cex = 0.7)
points(rep(2, length(x2)), x2, col = "red", pch = 19, cex = 0.7)
abline(h = mean_x1, col = "blue", lty = 2)
abline(h = mean_x2, col = "red", lty = 2)
legend("topright",
  legend = c(
    paste("Mean x1:", round(mean_x1, 2)),
    paste("Mean x2:", round(mean_x2, 2))
  ),
  col = c("blue", "red"), lty = 2, cex = 0.8
)
```



2. Consider different sampling schemes Argue for choice what is more natural and which advantages or disadvantages may apply.

If we want to compare each group (which is specified in the hypothesis H_1), then I think sampling with replacement from each group is more natural because this way we can see the differences between the groups better. If the two groups are considered to be very similar or we assume that they are approximately the same, then I think the other method (Centering both samples and then resample from the combined samples x_1 and x_2 for n_1 and n_2 times) is more natural (if we consider proving H_0).

3. Bootstrap using both strategies mentioned above using the t-test statistic. Calculate the bootstrap p-value based on 10000 bootstrap samples and 0.95 as well as 0.99 confidence intervals. Make your decision at the significance level 0.05 or 0.01, respectively.

```
bootstrap_t_test_independent <- function(x1, x2, num_bootstrap = 10000, alpha = 0.05) {
  observed_t_stat <- t.test(x1, x2, var.equal = FALSE)$statistic
  n1 <- length(x1)
  n2 <- length(x2)

  bootstrap_t_stats <- numeric(num_bootstrap)

  for (i in 1:num_bootstrap) {
    resampled_x1 <- sample(x1, size = n1, replace = TRUE)
    resampled_x2 <- sample(x2, size = n2, replace = TRUE)
    bootstrap_t_stats[i] <- t.test(resampled_x1, resampled_x2, var.equal = FALSE)$statistic
  }
}
```

```

}

p_value <- mean(abs(bootstrap_t_stats) >= abs(observed_t_stat))

ci_lower <- quantile(bootstrap_t_stats, alpha / 2)
ci_upper <- quantile(bootstrap_t_stats, 1 - alpha / 2)

list(
  observed_t_stat = observed_t_stat,
  p_value = p_value,
  confidence_interval = c(ci_lower, ci_upper)
)
}

bootstrap_t_test_pooled <- function(x1, x2, num_bootstrap = 10000, alpha = 0.05) {
  observed_t_stat <- t.test(x1, x2, var.equal = FALSE)$statistic
  n1 <- length(x1)
  n2 <- length(x2)

  x1_centered <- x1 - mean(x1)
  x2_centered <- x2 - mean(x2)
  pooled_data <- c(x1_centered, x2_centered)

  bootstrap_t_stats <- numeric(num_bootstrap)

  for (i in 1:num_bootstrap) {
    resampled_data <- sample(pooled_data, size = n1 + n2, replace = TRUE)
    resampled_x1 <- resampled_data[1:n1]
    resampled_x2 <- resampled_data[(n1 + 1):(n1 + n2)]
    bootstrap_t_stats[i] <- t.test(resampled_x1, resampled_x2, var.equal = FALSE)$statistic
  }

  p_value <- mean(abs(bootstrap_t_stats) >= abs(observed_t_stat))

  ci_lower <- quantile(bootstrap_t_stats, alpha / 2)
  ci_upper <- quantile(bootstrap_t_stats, 1 - alpha / 2)

  list(
    observed_t_stat = observed_t_stat,
    p_value = p_value,
    confidence_interval = c(ci_lower, ci_upper)
  )
}

results_independent <- bootstrap_t_test_independent(x1, x2, alpha = 0.05)
results_pooled <- bootstrap_t_test_pooled(x1, x2, alpha = 0.05)

results_independent_99 <- bootstrap_t_test_independent(x1, x2, alpha = 0.01)
results_pooled_99 <- bootstrap_t_test_pooled(x1, x2, alpha = 0.01)

results_independent

## $observed_t_stat
##          t
## -0.1188053

```

```
##
## $p_value
## [1] 0.9069
##
## $confidence_interval
##      2.5%      97.5%
## -2.573227  1.666590
```

```
results_pooled
```

```
## $observed_t_stat
##      t
## -0.1188053
##
## $p_value
## [1] 0.9092
##
## $confidence_interval
##      2.5%      97.5%
## -2.006542  2.001643
```

```
results_independent_99
```

```
## $observed_t_stat
##      t
## -0.1188053
##
## $p_value
## [1] 0.9114
##
## $confidence_interval
##      0.5%      99.5%
## -3.217042  2.208386
```

```
results_pooled_99
```

```
## $observed_t_stat
##      t
## -0.1188053
##
## $p_value
## [1] 0.9128
##
## $confidence_interval
##      0.5%      99.5%
## -2.569274  2.500543
```

Based we can reject H1 and say that there is no statistical significance between the two groups which means the results are consistent with the null hypothesis H0

4. Implement the corresponding permutation test and obtain p-value and confidence intervals as in 3. to get a corresponding test decision at the same significance levels.

```
permutation_t_test <- function(x1, x2, num_permutations = 10000, alpha = 0.05) {
  observed_t_stat <- t.test(x1, x2, var.equal = FALSE)$statistic
```

```

combined_data <- c(x1, x2)
n1 <- length(x1)
n2 <- length(x2)

perm_t_stats <- numeric(num_permutations)

for (i in 1:num_permutations) {
  permuted_data <- sample(combined_data)
  perm_x1 <- permuted_data[1:n1]
  perm_x2 <- permuted_data[(n1 + 1):(n1 + n2)]
  perm_t_stats[i] <- t.test(perm_x1, perm_x2, var.equal = FALSE)$statistic
}

p_value <- mean(abs(perm_t_stats) >= abs(observed_t_stat))
ci_lower <- quantile(perm_t_stats, alpha / 2)
ci_upper <- quantile(perm_t_stats, 1 - alpha / 2)

list(
  observed_t_stat = observed_t_stat,
  p_value = p_value,
  confidence_interval = c(ci_lower, ci_upper)
)
}

```

```

results_permutation <- permutation_t_test(x1, x2, alpha = 0.05)
results_permutation_99 <- permutation_t_test(x1, x2, alpha = 0.01)
results_permutation

```

```

## $observed_t_stat
##          t
## -0.1188053
##
## $p_value
## [1] 0.9143
##
## $confidence_interval
##      2.5%      97.5%
## -1.959767  1.920528

```

```
results_permutation_99
```

```

## $observed_t_stat
##          t
## -0.1188053
##
## $p_value
## [1] 0.9143
##
## $confidence_interval
##      0.5%      99.5%
## -2.558154  2.490972

```

We can see that the result doesn't change, we still reject H1

5. The Wilcoxon rank sum test statistic is the sum of ranks of the observations of sample 1 computed in the combined sample. Use bootstrapping with both strategies mentioned above and obtain p-value and confidence intervals as in 3. to get a corresponding test decision at the same significance levels.

```
bootstrap_wilcoxon_independent <- function(x1, x2, num_bootstrap = 10000, alpha = 0.05) {
  combined_data <- c(x1, x2)
  ranks <- rank(combined_data)
  observed_stat <- sum(ranks[1:length(x1)])

  n1 <- length(x1)
  n2 <- length(x2)
  bootstrap_stats <- numeric(num_bootstrap)

  for (i in 1:num_bootstrap) {
    resampled_x1 <- sample(x1, size = n1, replace = TRUE)
    resampled_x2 <- sample(x2, size = n2, replace = TRUE)
    combined_resample <- c(resampled_x1, resampled_x2)
    resample_ranks <- rank(combined_resample)
    bootstrap_stats[i] <- sum(resample_ranks[1:n1])
  }
  p_value <- mean(abs(bootstrap_stats - mean(bootstrap_stats)) >= abs(observed_stat - mean(bootstrap_stats)))
  ci_lower <- quantile(bootstrap_stats, alpha / 2)
  ci_upper <- quantile(bootstrap_stats, 1 - alpha / 2)

  list(
    observed_stat = observed_stat,
    p_value = p_value,
    confidence_interval = c(ci_lower, ci_upper)
  )
}

bootstrap_wilcoxon_pooled <- function(x1, x2, num_bootstrap = 10000, alpha = 0.05) {
  combined_data <- c(x1, x2)
  ranks <- rank(combined_data)
  observed_stat <- sum(ranks[1:length(x1)])

  n1 <- length(x1)
  n2 <- length(x2)
  bootstrap_stats <- numeric(num_bootstrap)

  pooled_data <- c(x1, x2)

  for (i in 1:num_bootstrap) {
    resampled_data <- sample(pooled_data, size = n1 + n2, replace = TRUE)
    resample_ranks <- rank(resampled_data)
    bootstrap_stats[i] <- sum(resample_ranks[1:n1])
  }
  p_value <- mean(abs(bootstrap_stats - mean(bootstrap_stats)) >= abs(observed_stat - mean(bootstrap_stats)))
  ci_lower <- quantile(bootstrap_stats, alpha / 2)
  ci_upper <- quantile(bootstrap_stats, 1 - alpha / 2)

  list(
```

```

        observed_stat = observed_stat,
        p_value = p_value,
        confidence_interval = c(ci_lower, ci_upper)
    )
}

results_wilcoxon_independent <- bootstrap_wilcoxon_independent(x1, x2, alpha = 0.05)
results_wilcoxon_pooled <- bootstrap_wilcoxon_pooled(x1, x2, alpha = 0.05)

results_wilcoxon_independent_99 <- bootstrap_wilcoxon_independent(x1, x2, alpha = 0.01)
results_wilcoxon_pooled_99 <- bootstrap_wilcoxon_pooled(x1, x2, alpha = 0.01)

results_wilcoxon_independent

## $observed_stat
## [1] 434
##
## $p_value
## [1] 1
##
## $confidence_interval
##      2.5%   97.5%
## 362.000 506.025

results_wilcoxon_pooled

## $observed_stat
## [1] 434
##
## $p_value
## [1] 0.6654
##
## $confidence_interval
##   2.5% 97.5%
##   378   524

results_wilcoxon_independent_99

## $observed_stat
## [1] 434
##
## $p_value
## [1] 1
##
## $confidence_interval
##   0.5% 99.5%
##   342   529

results_wilcoxon_pooled_99

## $observed_stat
## [1] 434
##
## $p_value
## [1] 0.6511
##

```

```
## $confidence_interval
##      0.5%      99.5%
## 357.4975 543.0000
```

6. Compare results to the results using t.test and wilcox.test

```
t_test_result <- t.test(x1, x2, var.equal = FALSE)
t_test_result
```

```
##
## Welch Two Sample t-test
##
## data: x1 and x2
## t = -0.11881, df = 23.027, p-value = 0.9065
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -0.9556081 0.8518000
## sample estimates:
## mean of x mean of y
## 0.2198182 0.2717222
```

```
wilcoxon_test_result <- wilcox.test(x1, x2)
wilcoxon_test_result
```

```
##
## Wilcoxon rank sum exact test
##
## data: x1 and x2
## W = 181, p-value = 0.6572
## alternative hypothesis: true location shift is not equal to 0
```

We can see that the results of the tests are similar to our results for the wilcoxon test but for the t test they are different.

Task 2

1. Create a sample of size 200 from the model above and for the independent predictor x3

```
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.4.2
```

```
n <- 200
x1 <- rnorm(n, mean = 2, sd = sqrt(3))
x2 <- runif(n, min = 2, max = 4)
x3 <- runif(n, min = -2, max = 2)
epsilon <- rt(n, df = 5)

y <- 3 + 2 * x1 + x2 + epsilon
data <- data.frame(x1 = x1, x2 = x2, x3 = x3, y = y)
kable(head(data), caption = "Generated sample for model y")
```

Table 1: Generated sample for model y

	x1	x2	x3	y
	-1.7407100	3.766241	-0.4566859	1.447831
	0.2167220	3.043631	0.9993084	6.998016
	4.1095652	3.132367	0.2753409	15.792749
	0.9681549	2.891732	1.7709891	8.989073
	1.7313932	2.410563	-0.2633439	9.584117
	0.9883816	2.818797	-1.4642166	8.233651

2. Do residual bootstrap for linear regression and fit the model $y = x_1 + x_2 + x_3$. Get the percentile CI for the coefficients. Can you exclude x_3 ?

Fit the linear regression model

```
model <- lm(y ~ x1 + x2 + x3, data = data)
```

Get the residuals

```
residuals <- resid(model)
fitted_values <- fitted(model)
```

Bootstrapping

```
B <- 1000 # number of bootstrap iterations

bootstrap_coefs <- matrix(NA, nrow = B, ncol = length(coef(model)))
colnames(bootstrap_coefs) <- names(coef(model))

for (i in 1:B) {
  resampled_residuals <- sample(residuals, replace = TRUE)
  new_y <- fitted_values + resampled_residuals
  new_model <- lm(new_y ~ x1 + x2 + x3, data = data)
  bootstrap_coefs[i, ] <- coef(new_model)
}
```

Calculate the confidence intervals and check if x_3 can be excluded

```
CI_lower <- apply(bootstrap_coefs, 2, function(x) quantile(x, probs = 0.025))
CI_upper <- apply(bootstrap_coefs, 2, function(x) quantile(x, probs = 0.975))

bootstrap_CI <- data.frame(
  Coefficient = names(coef(model)),
  Estimate = coef(model),
  CI_Lower = CI_lower,
  CI_Upper = CI_upper
)

kable(bootstrap_CI, caption = "Percentiles summary for residual bootstrap")
```

Table 2: Percentiles summary for residual bootstrap

	Coefficient	Estimate	CI_Lower	CI_Upper
(Intercept)	(Intercept)	2.7309466	1.8307320	3.7104389
x1	x1	1.9960812	1.8895766	2.0979798

	Coefficient	Estimate	CI_Lower	CI_Upper
x2	x2	1.0694943	0.7632738	1.3567193
x3	x3	0.0572073	-0.0937679	0.2005286

```
exclude_x3 <- (CI_lower["x3"] <= 0 && CI_upper["x3"] >= 0)
exclude_x3
```

```
## [1] TRUE
```

This means that x3 can be excluded from the model.

3. Do pairs bootstrap for linear regression and fit the model $y = x_1 + x_2 + x_3$. Get the percentile CI for the coefficients. Can you exclude x3?

We can use the previous base model we computed in 2. and the residuals from that model to do pairs bootstrapping

```
n <- nrow(data)
for (i in 1:B) {
  bootstrap_indices <- sample(1:n, n, replace = TRUE)
  bootstrap_data <- data[bootstrap_indices, ]
  new_model <- lm(y ~ x1 + x2 + x3, data = bootstrap_data)
  bootstrap_coefs[i, ] <- coef(new_model)
}

CI_lower <- apply(bootstrap_coefs, 2, function(x) quantile(x, probs = 0.025))
CI_upper <- apply(bootstrap_coefs, 2, function(x) quantile(x, probs = 0.975))

bootstrap_CI <- data.frame(
  Coefficient = names(coef(model)),
  Estimate = coef(model),
  CI_Lower = CI_lower,
  CI_Upper = CI_upper
)

kable(bootstrap_CI, caption = "Percentiles summary for pairs bootstrap")
```

Table 3: Percentiles summary for pairs bootstrap

	Coefficient	Estimate	CI_Lower	CI_Upper
(Intercept)	(Intercept)	2.7309466	1.8610975	3.4967034
x1	x1	1.9960812	1.9029814	2.0928572
x2	x2	1.0694943	0.8190554	1.3409222
x3	x3	0.0572073	-0.0933436	0.1971834

```
exclude_x3 <- (CI_lower["x3"] <= 0 && CI_upper["x3"] >= 0)
exclude_x3
```

```
## [1] TRUE
```

We can still exclude x3 from the model.

4. Compare the two approaches in 2. and 3. and explain the differences in the sampling approach and how this (might) affect(s) the results.

There are no major differences in the results between the two approaches, however, using only the residuals this method might not be as accurate as the pairs bootstrap which resamples the entire dataset and therefore the confidence intervals and the result might be more sensitive than the first approach.

Task 3

The bootstrapping methodology can be summarized as follows:

1. Resample the data: Given a sample of size n , randomly select n data points
2. Fit the model: compute the statistic of interest or estimate the parameter of interest
3. Repeat the process a number of times (in our cases 10000 times)
4. Compute the confidence intervals and reject or accept the hypothesis

Bootstrapping is a very useful method because its easy to implement and it helps provide confidence intervals which are reliable and accurate. Some disadvantages are that the method can be computationally heavy if there is a lot of data and a lot of samples to compute, and since it uses only observed data it might be biased and sometimes not represent the actual distribution.